

TLM: Deep Learning

Chapter 4: Word Embedding

Outline:

- (1) Word embedding – origins and fundamentals
- (2) Distributed representations
- (3) Static embeddings
- (4) Creating your own embeddings using Gensim
- (5) Exploring the embedding space with Gensim
- (6) Using word embeddings for spam detection
- (7) Neural embedding – not just for words
- (8) Character and subword embedding
- (9) Dynamic embeddings
- (10) Sentence and paragraph embeddings
- (11) Language-based model embeddings

Dosen Penyusun:

David Agustriawan, S.Kom., M.S.c., Ph.D.

Ajie Kusuma Wardhana, S.Kom., M.Eng

Word embedding – origins and fundamentals

Wikipedia defines **word embedding** as the collective name for a set of language modelling and feature learning techniques in **natural language processing (NLP)** where **words or phrases from a vocabulary are mapped to vectors of real numbers**. Deep learning model don't work directly with text. It needs to be converted to numbers instead. The process of converting text to numbers is a process called **vectorization**.

An early technique for vectorizing words was **one-hot encoding** however a major problem with it is that it treats each word as completely independent from all the others, since the similarity between any two words (measured by the dot product of two word vectors) is always zero. To overcome the limitation of one-hot encoding, the NLP has borrowed technique from **information retrieval (IR)** to vectorize text using the document as the context. Notable techniques are **term frequency-Inverse document frequency (TF-IDF)**, **latent semantic analysis (LSA)**, and **Topic modelling**. These representations attempt to capture a document-centric idea of semantic similarity between words.

Of these, **one-hot** and **TF-IDF** are **relatively sparse embeddings**, since vocabularies are usually quite large, and a word is unlikely to occur in more than a few documents in the corpus. The development **word embedding techniques** began around 2000. These techniques differ from previous IR-based techniques in that they use neighboring words as their context, leading to a more natural semantic similarity from a human understanding perspective.

Currently, word embedding is a foundational technique for all kinds of NLP tasks, such as text classification, document clustering, part of speech tagging, named entity recognition, sentiment analysis, and many more. Word embeddings result in dense, low dimensional vectors, and along with LSA and topic models can be thought of as a vector of latent features for the word. **Word embeddings are based on the distributional hypothesis, which states that words that occur in the similar contexts tend to have similar meanings.** Hence the class of word embedding based encodings is also known as distributed representations.

Distributed Representations

Distributed representations attempt to capture the meaning of a word by considering its relations with other words in its context. How does it work? Consider the following pair of sentences:

Paris is the capital of France

Berlin is the capital of Germany

Even assuming no knowledge of world geography, the sentence pair implies some sort of relationship between the entities Paris, France, Berlin, and Germany that could be represented as:

“Paris” is to “France” as “Berlin” is to “Germany.”

Distributed representations are based on the idea that there exists some transformation, as follows:

Paris : France :: Berlin : Germany

A distributed embedding space is one where words that are used in similar contexts are close to one another. Therefore, the similarity between the word vectors in this space would roughly correspond to the semantic similarity between the words.

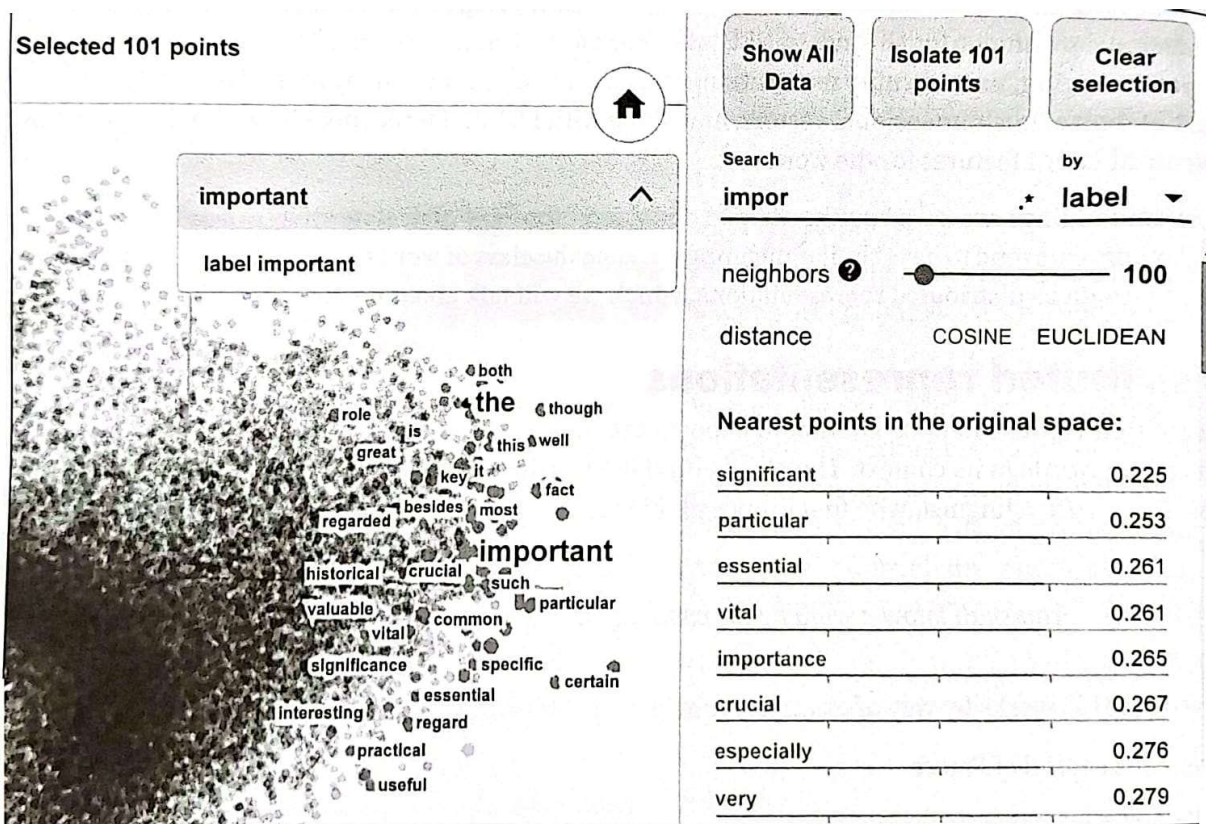


Figure 4.1: Visualization of nearest neighbors of the word “important” in a word embedding dataset, from the TensorFlow Embedding Guide (<https://www.tensorflow.org/guide/embedding>)

Figure 4.1 shows a TensorBoard visualization of word embedding of words around the word “important” in the embedding space. As you can see, the neighbors of the word tend to be closely related, or interchangeable with the original word. **For example, “crucial” is virtually a synonym, and it is easy to see how the words “historical” or “valuable” could be substituted in certain situations.**

Static embeddings

Static embeddings are the oldest type of word embedding. The embeddings are generated against a large corpus but the number of words, though large, is finite. It is like a dictionary, with words as the keys and their corresponding vector as the value. *If you have a word whose embedding needs to be looked up that was not in the original corpus, then you are out of luck. In addition, a word has the same embedding regardless of how it is used, so static embeddings cannot address the problem of polysemy, that is, words with multiple meanings.*

Word2Vec → word embedding for predictive model

The models known as Word2Vec were first created in 2013. The models are self-supervised, that is, they are supervised models that depend on the structure of natural language to provide labeled training data. The two architectures for Word2Vec are as follows:

- Continuous Bag of Words (CBOW)
- Skip-gram

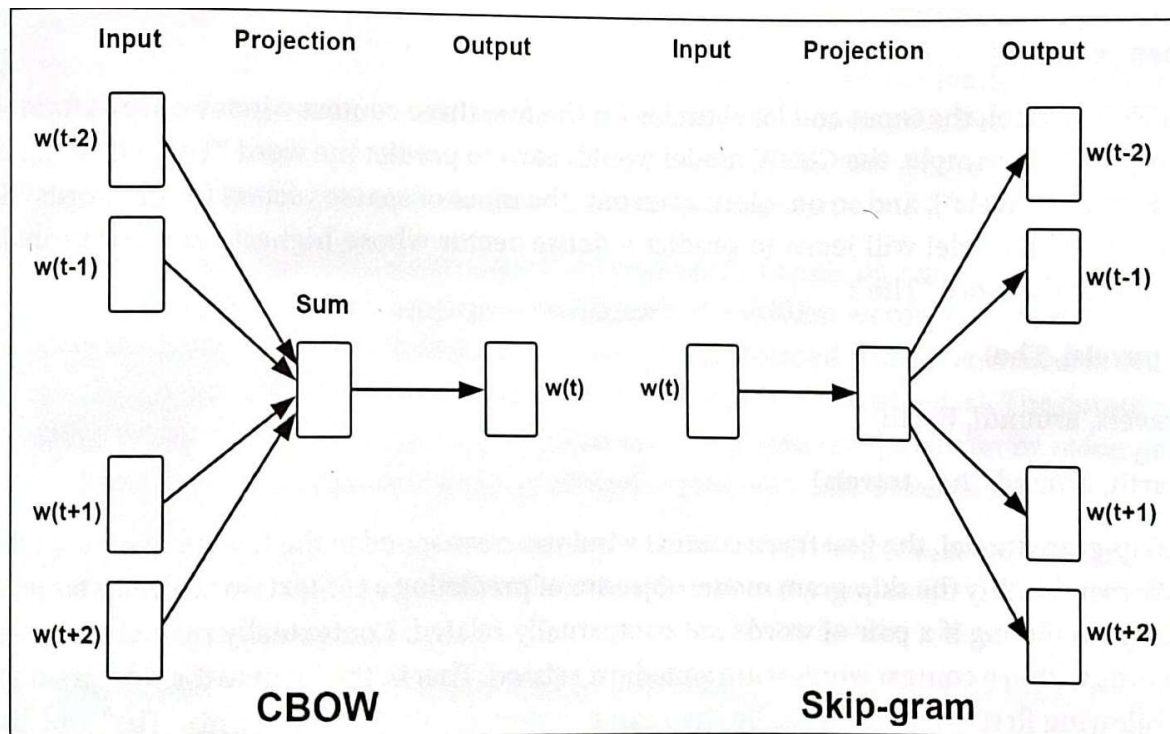


Figure 4.2: Architecture of the CBOW and Skip-gram Word2Vec models

In the **CBOW architecture**, the model predicts the current word given a window of surrounding words. The order of context words does not influence the prediction. In the **skip-gram architecture**, the model predicts the surrounding words given the context word. According to the Word2Vec website, CBOW is faster, but skip-gram does a better job at predicting infrequent words.

To understand the inputs and outputs, consider the following example sentence:

The Earth travels around the Sun once per year.

Assuming a **window size of 5**, that is, **two context words to the left and right of the content word**, the resulting context windows are shown as follows. The word in bold is the word under consideration, and the other words are the context words within the window:

[_, _ **The**, Earth, travels]

[_, The, **Earth**, travels, around]

[The, Earth, **travels**, around, the]

[Earth, travels, **around**, the, Sun]

[travels, around, **the**, Sun, once]

[around, the, **Sun**, once, per]

[the, Sun, **once**, per, year]

[Sun, **once**, per, year, _]

[**once**, per, year, _, _]

For the CBOW model, the input and label tuples for the first three context windows are as follows: **In the following first example, the CBOW model would learn to predict the word “The” given the set of words (“Earth,” “travels”), and so on.** More correctly, the input of sparse vectors for the words “Earth” and “travels”. The model will learn to predict a dense vector whose highest value, or probability, corresponds to the word “The”:

([Earth,travels], **The**)

([The, travels, around], **Earth**)

([The, earth, around, the], **travels**)

For the skip-gram model, the first three context windows correspond to the following input and label tuples. We can simplify the skip-gram model objective of predicting a context word given a target word to basically predicting if a pair of words are contextually related. Contextually related means that a pair of words within a context window are somehow related. That is, **the input to the skip-gram model for the following first example would be the sparse vectors for the context words “The” and “Earth,” and the output would be the value 1:**

([The, Earth], 1)

([The, travels], 1)

([Earth, The], 1)

([Earth, travels], 1)

([Earth, around], 1)

([travels, The], 1)
 ([travels, Earth], 1)
 ([travels, around], 1)
 ([travels, the], 1)

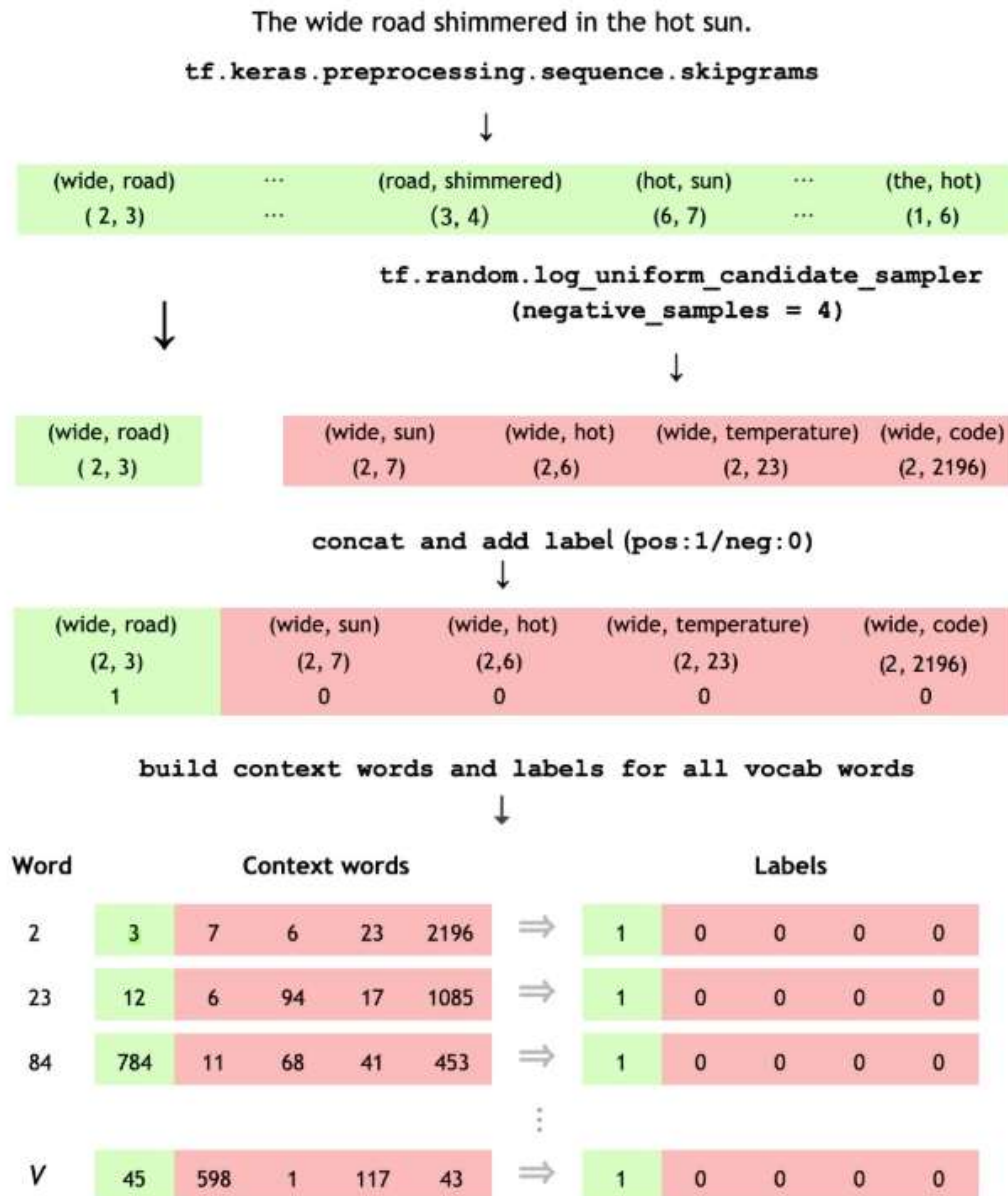
It is also need negative samples to train a model properly, **so it will generate additional negative samples by pairing each input word with some random word in the vocabulary. This process is called negative sampling** and might result in the following additional inputs:

([Earth, aardvark], 0)
 ([Earth, zebra], 0)

A model trained with all of these input is called a **Skip-Gram with Negative Sampling (SGNS) model**.

Window Size	Text	Skip-grams
2	[The wide road shimmered] in the hot sun.	wide, the wide, road wide, shimmered
	The [wide road shimmered in the] hot sun.	shimmered, wide shimmered, road shimmered, in shimmered, the
	The wide road shimmered in [the hot sun].	sun, the sun, hot
3	[The wide road shimmered in] the hot sun.	wide, the wide, road wide, shimmered wide, in
	[The wide road shimmered in the hot] sun.	shimmered, the shimmered, wide shimmered, road shimmered, in shimmered, the shimmered, hot
	The wide road shimmered [in the hot sun].	sun, in sun, the sun, hot

This diagram summarizes the procedure of generating a training example from a sentence:



Notice that the words temperature and code are not part of the input sentence. They belong to the vocabulary like certain other indices used in the diagram above.

It is important to understand that we are not interested in the ability of these models to classify; rather, we are interested in the side effect of training – the learned weights. The learned weights are what we call the embedding.

The Word2Vec model was trained in a self-supervised manner by Google on roughly 100 billion words from the Google News dataset and contains a vocabulary of 3 million words. Google then release the pretrained model for anyone to download and use. **The pretrained Word2Vec model is available at <https://mccormickml.com/2016/04/12/googles-pretrained-word2vec-model-in-python/>.** The output vector dimensionality is 300. It is available as a BIN file and can be opened using Gensim by using `gensim.models.Word2Vec.load_word2vec_format()` or using the `gensim()` data downloader.

Link to learn more: <https://www.tensorflow.org/text/tutorials/word2vec>.

GloVe → word embedding for count based model

The Global vectors for word representation (GloVe) is unsupervised learning algorithm for obtaining vector representation of words. Training is performed on aggregated global word-word co-occurrence statistic from corpus, and the resulting representations show similar clustering behaviour between similar words as seen in Word2Vec.

GloVe differs from Word2vec in that Word2vec is a predictive model while GloVe is a count-based model. The first step is to construct a large matrix of (word, context) pairs that co-occur in the training corpus. Rows correspond to words and column correspond to contexts, usually a sequence of one or more words. Each element of the matrix represents how often the word co-occurs in the context. The GloVe process factorize this co-occurrence matrix into a pair of (word, feature) and (feature, context) matrices. The process is known as matrix factorization and is done using Stochastic Gradient Descent (SGD), an iterative numerical method.

The GloVe process is much more resource-intensive than Word2Vec. This is because Word2vec learns the embedding by training over batches of word vectors, while GloVe factorizes the entire co-occurrence matrix in one shot. In order to make the process scalable, SGD is often used in parallel mode. GloVe vectors trained on various large corpora (number of tokens ranging from 6 billion to 840 billion, vocabulary size from 400 thousand to 2.2 million) and of various dimensions (50, 100, 200, 300) are available from the GloVe project download page <https://nlp.stanford.edu/projects/glove/>.

Creating your own embeddings using Gensim

We will create an embedding using Gensim and a small text corpus, called text8.

Gensim is an open-source Python library designed to extract semantic meaning from text documents. One of its feature is an excellent implementation of the Word2Vec algorithm, with an easy to use API that allows you to train and query your own Word2Vec model. To install Gensim, please follow the instructions at <https://radimrehurek.com/gensim/install.html>

The text8 dataset is the first 10^8 bytes of the Large Text Compression Benchmark, which consist of the first 10^9 bytes of English wikipedia. The text8 dataset is accessible from within the Gensim API as an iterable of tokens, essentially a list of tokenized sentences. To download the text8 corpus, create a Word2Vec model from it, and save it for later use, run the following few lines of code:

```
import gensim.downloader as api
from gensim.models import Word2Vec
dataset = api.load("text8")
model = Word2Vec(dataset)
model.save("data/text8-word2vec.bin")
```

This will train a Word2vec model on the text8 dataset and save it as a binary file. The Word2Vec model has many parameters, but we will just use the defaults. In this case, it trains a CBOW model (sg=0) with window size 5 (window=5) and will produce 100 dimensional embeddings (size = 100). The full set of parameters is described on the **Word2Vec documentation page**. To run this code execute the following commands at the command line:

```
$ mkdir data
$ python create_embedding_with_text8.py
```

The code should run for 5-10 minutes, after which it will write out a trained model into the data folder.

The online Tensorflow tutorial contains an example of how to train a Word2Vec model from scratch (<https://www.tensorflow.org/tutorials/text/word2vec>).

Exploring the embedding space with Gensim

Let us reload the Word2Vec model we just built and explore it using the Gensim API. The actual word vectors can be assessed as a custom Gensim class from the model's `wv` attribute:

```
from gensim.models import KeyedVectors
model = KeyedVectors.load("data/text8-word2vec.bin")
word_vectors = model.wv
```

We can take a look at the first few words in the vocabulary and check to see if specific words are available:

```
words = word_vectors.vocab.keys()
print([x for i, x in enumerate(words) if i < 10])
assert("king" in words)
```

The preceding snippet of code produces the following output:

```
['anarchism', 'originated', 'as', 'a', 'term', 'of', 'abuse', 'first', 'used',
'against']
```

We can look for similar words to a give word("king"), shown as follows:

```
def print_most_similar(word_conf_pairs, k):
    for i, (word, conf) in enumerate(word_conf_pairs):
        print("{:.3f} {}".format(conf, word))
        if i >= k-1:
            break
    if k < len(word_conf_pairs):
        print("...")
print_most_similar(word_vectors.most_similar("king"), 5)
```

The most_similar() method with a single parameter produce the following output. Here, the floating-point score is a measure of the similarity, higher values being better than lower values, as you can see, the similar words seem to be mostly accurate:

```
0.760 prince
0.701 queen
0.700 kings
0.698 emperor
0.688 throne
...
```

You can also do vector arithmetic similar to the country-capital example previously with the objective to see if the relation Paris : France :: Berlin : Germany holds true. This is equivalent to saying that the distance in embedding space between Paris and France should be the same as that between Berlin and Germany. In the other words, France – Paris + Berlin should give us Germany.

```
print_most_similar(word_vectors.most_similar(
    positive=["france", "berlin"], negative=["paris"]), 1
)
```

This returns the following result, as expected:

```
0.803 germany
```

The preceding similarity value reported is cosine similarity, but a better similarity proposed by Levy and Goldberg which is also implemented in the Gensim API. This measure essentially computes the distance on a log scale thereby amplifying the difference between shorter distances and reducing the difference between longer ones:

```
print_most_similar(word_vectors.most_similar_cosmul(
    positive=["france", "berlin"], negative=["paris"]), 1
)
```

And this also yields the expected result, but with higher similarity:

```
0.984 germany
```

Gensim also provides a `doesnt_match()` function, which can be used to detect the odd one out of a list of words:

```
print(word_vectors.doesnt_match(["hindus", "parsis", "singapore",  
                                "christians"]))
```

This gives us `singapore` as expected, since it is the only country among a set of words identifying religions.

We can also calculate the similarity between two words. Here we demonstrated that the distance between related words is less than that of unrelated words:

```
for word in ["woman", "dog", "whale", "tree"]:  
    print("similarity({:s}, {:s}) = {:.3f}".format(  
        "man", word,  
        word_vectors.similarity("man", word)  
    ))
```

This gives the following interesting result:

```
similarity(man, woman) = 0.759  
similarity(man, dog) = 0.474  
similarity(man, whale) = 0.290  
similarity(man, tree) = 0.260
```

The `similar_by_word()` function is functionally equivalent to `similar()` except that the latter normalizes the vector before comparing by default. There is also a related `similar_by_vector()` function, which allows you to find similar words by specifying a vector as input. Here we try to find words that are similar to "singapore":

```
print(print_most_similar(  
    word_vectors.similar_by_word("singapore"), 5)  
)
```

And we get the following output, which seems to be mostly correct, at least from a geographical point of view:

```
0.882 malaysia  
0.837 indonesia  
0.826 philippines  
0.825 uganda  
0.822 thailand  
...
```

Using word embeddings for spam detection

Text is treated as a sequence of tokens. The embedding provides a dense fixed dimension vector for each token. Each token is replaced with its vector, and this converts the sequence of text into a matrix of examples, each of which has a fixed number of features corresponding to the dimensionality of the embedding. This matrix of example can be used directly as input to standard (non-neural network based) machine learning programs or in this case we can use one-dimensional version of the CNN.

The project example is a spam detector that will classify Short Message Service (SMS) or text message as either “ham” or “spam”. In here we will see how the program learns an embedding from scratch that is customized to the spam detection task:

Full code: in this link <https://github.com/PacktPublishing/Deep-Learning-with-TensorFlow-and-Keras-3rd-edition>. Go to chapter 4, spam classifier:

As usual, we will start with our imports:

```
import argparse
import gensim.downloader as api
import numpy as np
import os
import shutil
import tensorflow as tf
from sklearn.metrics import accuracy_score, confusion_matrix
```

Getting the data

The data is a public data comes from the SMS spam collection dataset from the UCI machine learning repository. The following code will download the file and parse it to produce a list of SMS messages and their corresponding labels:

```
def download_and_read(url):
    local_file = url.split('/')[-1]
    p = tf.keras.utils.get_file(local_file, url,
                                extract=True, cache_dir=".")
    labels, texts = [], []
    local_file = os.path.join("datasets", "SMSSpamCollection")
    with open(local_file, "r") as fin:
        for line in fin:
            label, text = line.strip().split('\t')
            labels.append(1 if label == "spam" else 0)
            texts.append(text)
    return texts, labels
DATASET_URL = "https://archive.ics.uci.edu/ml/machine-learning-databases/00228/smsspamcollection.zip"
texts, labels = download_and_read(DATASET_URL)
```

The dataset contains 5,574 SMS records, 747 of which are marked as “spam” and the other 4,827 are marked as “ham” (not spam). The text of the SMS records is contained in the variable `texts`, and the corresponding numeric labels (0 = ham, 1 = spam) are contained in the variable `labels`.

Making the data ready for use

The SMS text needs to be fed into the network as a sequence of integers, where each word is represented by its corresponding ID in the vocabulary. We will **use the keras tokenizer to convert each SMS text into a sequence of words, and then create the vocabulary using the fit_on_text() method on the tokenizer.** We then convert the SMS message to a sequence of integers using `text_to_sequence()`. Finally, since the network can only work with fixed-length sequence of integers, we call the `pad_sequences()` function to **pad the shortest SMS message with zeros.** The longest SMS message in the dataset has 189 tokens (words). In many applications where there may be a few outlier sequences that are very long, so we should restrict the length to a smaller number by **setting the maxlen flag.** In that case, **sentences longer than maxlen tokens would be truncated, and sentences shorter than maxlen tokens would be padded:**

```
# tokenize and pad text
tokenizer = tf.keras.preprocessing.text.Tokenizer()
tokenizer.fit_on_texts(texts)
text_sequences = tokenizer.texts_to_sequences(texts)

text_sequences = tf.keras.preprocessing.sequence.pad_sequences(
    text_sequences)
num_records = len(text_sequences)
max_seqlen = len(text_sequences[0])
print("{:d} sentences, max length: {:d}".format(
    num_records, max_seqlen))
```

We will also **convert our labels to categorical or one-hot encoding format**, because the loss function we would like to choose (categorical cross-entropy) expects to see the labels in that format:

```
# Labels
NUM_CLASSES = 2
cat_labels = tf.keras.utils.to_categorical(
    labels, num_classes=NUM_CLASSES)
```

The tokenizer allows access to the vocabulary created through the `word_index` attribute, which is basically a dictionary of vocabulary words to their index positions in the vocabulary. We also build the reverse index that enables us to go from index position to word itself, in addition, we create entries for the PAD character:

```
# vocabulary
word2idx = tokenizer.word_index
idx2word = {v:k for k, v in word2idx.items()}
word2idx["PAD"] = 0
idx2word[0] = "PAD"
vocab_size = len(word2idx)
print("vocab size: {:d}".format(vocab_size))
```

Finally, we create the dataset object that our network will work with. The dataset object allows us to set up some properties, such as the batch size, declaratively. **Here, we build up a dataset from our padded sequence of integers and categorical labels**, shuffle the data, and split it into training, validation, and test sets. Finally, we set the batch size for each of the three datasets:

```
# dataset
dataset = tf.data.Dataset.from_tensor_slices(
    (text_sequences, cat_labels))
dataset = dataset.shuffle(10000)
test_size = num_records // 4
val_size = (num_records - test_size) // 10
test_dataset = dataset.take(test_size)
val_dataset = dataset.skip(test_size).take(val_size)
train_dataset = dataset.skip(test_size + val_size)
BATCH_SIZE = 128
test_dataset = test_dataset.batch(BATCH_SIZE, drop_remainder=True)
val_dataset = val_dataset.batch(BATCH_SIZE, drop_remainder=True)
train_dataset = train_dataset.batch(BATCH_SIZE, drop_remainder=True)
```

Building the embedding matrix

The Gensim toolkit provides access to various trained embedding models, as you can see from running the following command at the python prompt:

```
>>> import gensim.downloader as api
>>> api.info("models").keys()
```

This will return the following **trained word embeddings**:

- **Word2Vec**: Two flavors, one trained on Google news (3 million word vectors based on 3 billion tokens), and one trained on Russian corpora (word2vec-ruscorpora-300, word2vec-google-news-300).
- **GloVe**: Two flavors, one trained on the Gigawords corpus (400,000 word vectors based on 6 billion tokens), available as 50d, 100d, 200d, and 300d vectors, and one trained on Twitter (1.2 million word vectors based on 27 billion tokens), available as 25d, 50d, 100d, and 200d vectors (glove-wiki-gigaword-50, glove-wiki-gigaword-100, glove-wiki-gigaword-200, glove-wiki-gigaword-300, glove-twitter-25, glove-twitter-50, glove-twitter-100, glove-twitter-200). Smaller embedding sizes would result in greater compression of the input and consequently a greater degree of approximation.
- **fastText**: One million word vectors trained with subword information on Wikipedia 2017, the UMBC web corpus, and statmt.org news dataset (16B tokens) (fastText-wiki-news-subwords-300).
- **ConceptNet Numberbatch**: An ensemble embedding that uses the ConceptNet semantic network, the paraphrase database (PPDB), Word2Vec, and GloVe as input. Produces 600d vectors [12, 13].

For our example, we chose the 300d GloVe embeddings trained on the gigaword corpus. In order to keep our model size small, we want to only consider embeddings for words that exist in our vocabulary. This is done using the following code, which creates a smaller embedding matrix for each word in the vocabulary. Each row in the matrix corresponds to a word, and the row itself is the vector corresponding to embedding for the word:

```
def build_embedding_matrix(sequences, word2idx, embedding_dim,
                           embedding_file):
    if os.path.exists(embedding_file):
        E = np.load(embedding_file)
    else:
        vocab_size = len(word2idx)
        E = np.zeros((vocab_size, embedding_dim))
        word_vectors = api.load(EMBEDDING_MODEL)
        for word, idx in word2idx.items():
            try:
                E[idx] = word_vectors.word_vec(word)
            except KeyError: # word not in embedding
                pass
```

```

        np.save(embedding_file, E)
    return E
EMBEDDING_DIM = 300
DATA_DIR = "data"
EMBEDDING_NUMPY_FILE = os.path.join(DATA_DIR, "E.npy")
EMBEDDING_MODEL = "glove-wiki-gigaword-300"
E = build_embedding_matrix(text_sequences, word2idx,
    EMBEDDING_DIM,
    EMBEDDING_NUMPY_FILE)
print("Embedding matrix:", E.shape)

```

Defining the spam classifier

We are now ready to define our classifier. We will use a **one-dimensional convolutional neural network or ConvNet (1D CNN)**. The input is a sequence of integers. The first layer is an embedding layer which convert each input integers to a vector of size (**embedding_dim**). Depending on the run mode, that is, whether we will learn the embeddings from scratch, do transfer learning, or do fine-tuning, the embedding layer in the network would be slightly different. **When the network starts with randomly initialized embedding weights (run_mode == "scratch") and learns the weight during the training, we set the trainable parameter to True. In the transfer learning case (run_mode == "vectorizer"), we set the weights from our embedding matrix E but the trainable parameter to False, so it does not train. In the fine tuning case (run_mode == "finetuning") we set the embedding weights from our external matrix E, as well as setting the layer to trainable.**

The output of the embedding is fed into a convolutional layer. Here, fixed-size 3-token-wide 1D windows (**kernel_size=3**), also called time steps, are convolved against 256 random filters (**num_filters = 256**) to produce vector of size 256 for each time step. Thus, the output vector of shape is (**batch_size, time_steps, num_filters**)

The output of the convolutional layer is sent to 1D spatial dropout layer. **Spatial dropout will randomly drop entire feature maps output from the convolutional layer.** This is a regularization technique to prevent overfitting. **This is then sent through a global max pool layer**, which takes the maximum value from each time step for each filter, resulting in a vector of shape (**batch_size, num_filters**).

The output of the dropout layer is fed into a pooling layer to flatten it, and then into a dense layer, which convert a vector of shape (**batch_size, num_filters**) to (**batch_size, num_classes**). A softmax activation will convert the scores for each of (spam, ham) into a probability distribution, indicating the probability of the input SMS being spam or ham respectively:

```

class SpamClassifierModel(tf.keras.Model):
    def __init__(self, vocab_sz, embed_sz, input_length,
                 num_filters, kernel_sz, output_sz,
                 run_mode, embedding_weights,
                 **kwargs):
        super(SpamClassifierModel, self).__init__(**kwargs)
        if run_mode == "scratch":
            self.embedding = tf.keras.layers.Embedding(vocab_sz,
                                                       embed_sz,
                                                       input_length=input_length,
                                                       trainable=True)
        elif run_mode == "vectorizer":
            self.embedding = tf.keras.layers.Embedding(vocab_sz,
                                                       embed_sz,
                                                       input_length=input_length,
                                                       weights=[embedding_weights],
                                                       trainable=False)
        else:
            self.embedding = tf.keras.layers.Embedding(vocab_sz,
                                                       embed_sz,
                                                       input_length=input_length,
                                                       weights=[embedding_weights],
                                                       trainable=True)
        self.conv = tf.keras.layers.Conv1D(filters=num_filters,
                                             kernel_size=kernel_sz,
                                             activation="relu")
        self.dropout = tf.keras.layers.SpatialDropout1D(0.2)
        self.pool = tf.keras.layers.GlobalMaxPooling1D()
        self.dense = tf.keras.layers.Dense(output_sz,
                                             activation="softmax")
    def call(self, x):
        x = self.embedding(x)
        x = self.conv(x)
        x = self.dropout(x)
        x = self.pool(x)
        x = self.dense(x)
        return x
# model definition
conv_num_filters = 256
conv_kernel_size = 3
model = SpamClassifierModel(
    vocab_size, EMBEDDING_DIM, max_seqlen,
    conv_num_filters, conv_kernel_size, NUM_CLASSES,
    run_mode, E)
model.build(input_shape=(None, max_seqlen))

```

Finally, we compile the model using the categorical cross entropy loss function and the Adam optimizer:

```

# compile
model.compile(optimizer="adam", loss="categorical_crossentropy",
              metrics=["accuracy"])

```

Training and evaluating the model:

Since the dataset is imbalanced; there are only 747 instances of spam compared to 4,827 instances of ham. **The network could achieve close to 87% accuracy simply by always predicting the majority class. To alleviate this problem, we set class weights to indicate that an error on a spam SMS is eight times as expensive as an error on a ham SMS. This is indicated by the CLASS_WEIGHT variable, which is passed into the model.fit() calls as an additional parameter.**

After training for 3 epochs, we evaluate the model against the test set and report the accuracy and confusion matrix of the model against the test set. **However, for imbalanced dataset, even with the use of class weight, the model may end up learning to always predict the majority class. Therefore it is generally advisable to report accuracy on a per class basis to make sure that the model learns to distinguish each class effectively.** This can be done quite easily using the confusion matrix by dividing the diagonal element for each row by the sum of elements for that row, where each row corresponds to a labelled class:

```
NUM_EPOCHS = 3
# data distribution is 4827 ham and 747 spam (total 5574), which
# works out to approx 87% ham and 13% spam, so we take reciprocals
# and this works out to being each spam (1) item as being
# approximately 8 times as important as each ham (0) message.
CLASS_WEIGHTS = { 0: 1, 1: 8 }
# train model
model.fit(train_dataset, epochs=NUM_EPOCHS,
          validation_data=val_dataset,
          class_weight=CLASS_WEIGHTS)
# evaluate against test set
labels, predictions = [], []
for Xtest, Ytest in test_dataset:
    Ytest_ = model.predict_on_batch(Xtest)
    ytest = np.argmax(Ytest, axis=1)
    ytest_ = np.argmax(Ytest_, axis=1)
    labels.extend(ytest.tolist())
    predictions.extend(ytest_.tolist())

print("test accuracy: {:.3f}".format(accuracy_score(labels, predictions)))
print("confusion matrix")
print(confusion_matrix(labels, predictions))
```

Running the spam detector:

The three scenarios we want to look at are:

- Letting the network learn the embedding for the task.
- Starting with a fixed external third-party embedding where the embedding matrix is treated like a vectorizer to transform the sequence of integers into a sequence of vectors.
- Starting with an external third-party embedding which is further fine-tuned to the task during the training.

Each scenario can be evaluated by setting the value of the mode argument as shown in the following command:

```
$ python spam_classifier --mode [scratch|vectorizer|finetune]
```

The dataset is small, and the model is fairly simple. We were able to achieve very good results (validation set accuracies in the high 90s, and perfect test set accuracy) with only minimal training (3 epochs). In all three cases, the network achieved a perfect score, accurately predicting the 1,111 ham messages, as well as the 169 spam cases.

The change in validation accuracies, shown in figure 4.3, illustrates the differences between the three approaches:

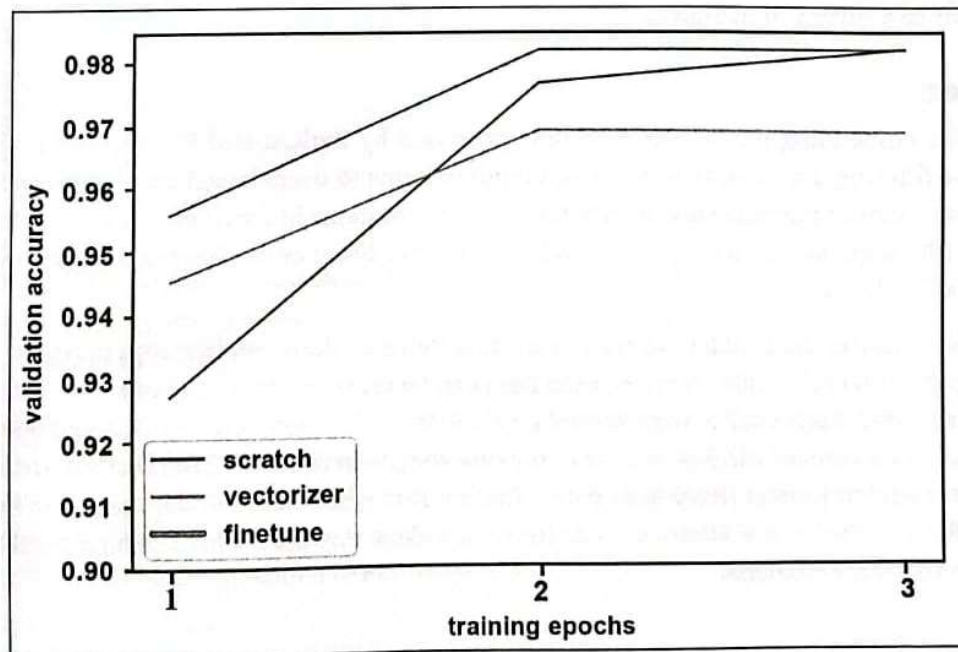


Figure 4.3: Comparison of validation accuracy across training epochs for different embedding techniques

Neural embedding not just for words

The application of word embeddings to non-word setting, also known as **neural embeddings**. As you will recall, **word embeddings leverage the distributional hypothesis that words occurring in similar contexts tend to have similar meanings, where context is usually a fixed-size (in number of words) window around the target word.**

The idea of neural embeddings is very similar; that is, entities that occur in similar contexts tend to be strongly related to each other. The way in which these context are constructed is usually situation-dependent.

Item2Vec

Item2Vec embedding model is **built for a collaborative filtering use case, that is, recommending items to users based on purchase by other users that have similar purchase histories to this user.** It uses items in a web-store as the “words” and the itemset (the sequence of items purchased by a user over time) as the “sentence” from which the “word context” is derived.

For example, consider the problem of recommending items to shoppers in a supermarket. Assume that our supermarket sells 5,000 items, so each item can be represented as a sparse one-hot encoded vector size 5,000. Each user is represented by their shopping cart, which is a sequence of such vectors. Applying a context window similar to the one we saw in the Word2Vec section, we can train a skip-gram model to predict likely item pairs. The learned embedding model maps the items to a dense low-dimensional space where similar items are close together, which can be used to make similar item recommendation.

Node2Vec

The Node2Vec embedding model is built as a scalable way to learn feature for nodes in a graph. It learns an embedding of the structure of the graph by executing a large number of fixed-length random walks on the graph. The nodes are the “words” and the random walks are the “sentences” from which the “word context” is derived in node2vec.

Example we will use a predecessor graph-based embedding called DeepWalk. To the dataset of 11,463 x 5,812 matrix of word counts, where the row represents words, and columns represent conference papers. We will use this to construct a graph of papers, where an edge between two papers represents a word that occurs in both of them. Both node2vec and deepwalk assume that the graph is undirected and unweighted. Our graph is undirected, since relationship between pair of papers is bidirectional. However, our edges could have weights based on the number of word co-occurrences between the two documents. For our example, we will consider any number of co-occurrences above 0 to be a valid unweighted edge.

Full code: https://github.com/PacktPublishing/Deep-Learning-with-TensorFlow-and-Keras-3rd-edition/tree/main/Chapter_4. Go to `neurips_papers_node2vec.py`.

Character and subword embeddings

Some key advantages of **character-level embeddings** over word embeddings:

First, a character vocabulary is finite and small: a vocabulary for English would contain around 70 characters (26 characters, 10 numbers, and the rest special characters)

Second, unlike word embeddings, which provide vectors for a large but finite set of words, there is no concept of out-of-vocabulary for character embeddings, since any word can be represented by the vocabulary.

Third, character embeddings tend to be better for rare and misspelled words because there is much less imbalance for character inputs than for word inputs.

Character embeddings tend to work better for applications that require the notion of syntactic (spelling, morphology, and word structure) rather than semantic similarity (meaning and concept). However, unlike word embeddings, character embeddings tend to be task-specific and are usually generated inline within a network to support the task. For this reason, third-party character embeddings are generally not available.

Subword embeddings combine the idea of character and word embeddings by treating a word as a bag of character n-grams, that is, sequence of n consecutive words.

For example, character n-grams for n=3 for the word “green” would be “<gr”, “gre”, “ree”, “een”, and “en>”. The beginning and end of words are marked with “<” and “>” characters respectively, to distinguish between short words and their n-grams such as “<cat>” and “cat”

Dynamic embeddings

Static embeddings are deployed as a dictionary of words (and subwords) mapped to fixed dimensional vectors. The vector corresponding to a word in these embeddings is going to be the same regardless of whether it is being used as a noun or verb in the sentence. For example: the word “ensure” (the name of a health supplement when used as a noun, and to make certain when used as a verb). It also provides the same vector for polysemous words or words with multiple meanings, such as “bank” (which can mean different things depending on whether it co-occurs with the word “money” or “river”). In both cases, the meaning of the word changes depending on clues available in its context, the sentence.

Dynamic embeddings attempt to use these signals to provide different vectors for words based on their context.

Dynamic embeddings are deployed as trained networks that convert your input (typically a sequence of one-hot vectors) into a lower-dimensional dense fixed-size embedding by looking at the entire sequence, not just individual words. You can either preprocess your input to this dense embedding and then use this as input to your task specific network, or wrap the network and treat it similar to the `tf.keras.layers.Embedding` layer for static embeddings.

The earliest dynamic embedding was called **Contextualized Vectors (CoVe)**. This involved taking the output of the encoder from the encoder-decoder pair of a machine translation network and concatenating it with word vectors for the same word.

Another dynamic embedding was **Embeddings from language Models (ELMo)**. ELMo computes contextualized word representations using character-based word representation and bidirectional long short term memory (LSTM). Here, an array of sentences will be used, where the **trained ELMo network model** will figure out tokens by using its default strategy of tokenizing on white space:

```
Import tensor flow as tf
```

```
Import tensorflow_hub as hub
```

```
elmo = hub.load("https://tfhub.dev/google/elmo/3")
```

```
embeddings = elmo.signatures["default"] (
```

```
    tf.constant ([
```

```
        "I like green eggs and ham",
```

```
        "would you eat them in a box"
```

```
    ]))["elmo"]
```

```
print (embedding.shape)
```

The output is (2, 7, 1024).

the first index tells us that our input contained 2 sentences. The second index refers to the maximum number of words across all sentences, in this case 7. The model automatically pads the output to the longest sentence. The third index gives us the size of the contextual word embedding created by ELMo; each word is converted to a vector of size (1024).

Sentence and paragraph embeddings

Convenient source of sentence embeddings is the Google Universal Sentence Encoder. As with ELMo, the google universal sentence encoder can also be loaded from TensorFlow Hub into your TF2 code.

```
embed = hub.load ("https://tfhub.dev/google/universal-sentence-encoder-large/4")
```

```
embeddings = embed([
```

```
    "i like green eggs and ham",
```

```
    "would you eat them in a box"
```

```
]) ["outputs"]
```

```
print(embeddings.shape)
```

The output is (2, 512)

That is, each sentence is represented by a vector size (512). It is important to note that the Google Universal Sentence Encoder can handle any length of word sequence – so you could legitimately use it to get word embeddings on one end as well as paragraph embeddings on the other. However, as the sequence length gets longer, the quality of the embeddings tends to get “diluted”.

Language model-based embeddings

A language model is a probability distribution over sequence of words. Once we have the model, we can ask it to predict the most likely next word given a particular sequence of words. Simialr to **traditional word embeddings, both static and dynamic**, they are trained to predict the next word (or previous word as well, if the language model is bidirectional) given a partial sentence from the corpus. **Training does not involve active labelling, since it leverages the natural grammatical structure of large volumes of text, so in a sense, this is a self-supervised learning process.**

The main difference between a language model as a word embedding and more traditional embeddings is that **traditional embeddings are applied as a single initial transformation on the data and are then fine-tuned for specific tasks.** In contrast, **language models are trained on large external corpora and represent a model of a particular language, say English. This step is called pretraining. The next step is to fine-tune these general-purpose language models for your particular application domain.**

You have accurately captured the core conceptual shift between **traditional word embeddings** (like Word2Vec or GloVe) and **modern language models** (like BERT, GPT, or RoBERTa).

Here is a breakdown of how these two approaches compare, adding a bit of technical context to your summary:

Direct Comparison

Feature	Traditional Embeddings (e.g., Word2Vec, GloVe)	Language Models / Contextual Embeddings (e.g., BERT, GPT)
Context Sensitivity	Static. The word "bank" has the exact same vector representation whether it means a river bank or a financial bank.	Dynamic. The vector representation changes depending on the surrounding words in the sentence.
Initial Role	Applied as a single initial transformation (a lookup table) at the very start of the neural network pipeline.	Serves as the entire backbone network that processes the text through multiple deep layers.
Knowledge Base	Captures semantic similarity (e.g., "king" is close to "queen") based on co-occurrence statistics.	Represents a deep understanding of syntax, grammar, and facts learned during extensive pretraining.
Adaptation	The weights of the static lookup table are either frozen or slightly modified during task training.	The entire pre-trained model (or specific layers) is fine-tuned on domain-specific data.

For example, if you are working in the travel or healthcare industry, you would fine-tuned the langauage model with text from your own domain. Fine-tuning involves retraining the las few layers with your own text. Once fine-tuned, you can reuse this model for multiple tasks within your domain. The fine-tuning

step is generally much less expensive compared to the pretraining step. Once you have the fine-tuned language model, you remove the last layer of the language model and replace it with one-to-two-layer fully connected network that converts the language model embedding for your input into the final categorical or regression output that your task needs.

The language model-based embedding idea has its roots in the ELMo network, which you have already seen in this chapter. ELMo learns about its language by being trained on a large text corpus to learn to predict the next and previous words given a sequence of words. ELMo is based on a bidirectional LSTM. Meanwhile, **the transformer architecture has become the preferred network for machine translation tasks, replacing the LSTM network because it allows for parallel operations and better handling of long-term dependencies.** Decoder stack from standard transformer network is used to build a language model embedding called **Generative Pretraining (GPT)** that achieved state of the art results for many language processing tasks. The paper proposes several configurations for supervised tasks involving single- and multi-sentence tasks such as classification, entailment, similarity, and multiple-choice question answering. The OpenAI team later followed this up by building even larger language models called **GPT-2** and **GPT-3** respectively. One problem with OpenAI transformer architecture is that it is unidirectional whereas its predecessors ELMo and ULMFiT were bidirectional.

Bidirectional Encoder Representation for Transformer (BERT) uses the encoder stack of the transformer architecture and achieves bidirectionality safely by masking up to 15% of its input, which it asks the model to predict. BERT proposes configurations for using it to several supervised learning tasks such as single- and multiple-sentence classification, question answering, and tagging.

BERT model comes in two major flavors – BERT-base and BERT-large. BERT-base has 12 encoder layers, 768 hidden units, and 12 attention heads, with 10 million parameters in all. BERT-large has 24 encoder layers, 1,024 hidden units, and 16 attention heads, with 340 million parameters.

BERT pretraining is an expensive process and can currently only be achieved using **Tensor Processing Units (TPUs)** or large distributed **Graphics Processing Units (GPUs)** clusters.

Using BERT as a feature extractor

The BERT project provides a set of python scripts that can be run from the command line to fine-tune BERT:

```
$ git clone https://github.com/google-research/bert.git
```

```
$ cd bert
```

We then download the appropriate BERT model we want to fine-tune. BERT-base and BERT-large, each has a cased and uncased version. The cased version differentiates between upper and lowercase words, while the uncased version does not. For our example, we will use the BERT-base-uncased pretrained model. You can find the download URL for this and other models further down the readme.md page:

```
$ mkdir data
```

```
$ cd data
```

```
$ wget \ https://storage.googleapis.com/bert\_models/2018\_10\_18/uncased\_L-12\_H-768\_A-12.zip
```

```
$ unzip -a uncased_L-12_H-768_A-12.zip
```

This will create the following folder under the data directory of your local BERT project. The bert_config.json file is the configuration file used to create the original pretrained model, and the vocab.txt is the vocabulary used for the model, consisting of 30,522 words and word pieces:

```
Uncased_L-12_H-768_A-12/
```

```
|----- bert_config.json
```

```
|----- bert_model.ckpt.data-00000-of-00001
```

```
|----- bert_model.ckpt.index
```

```
|----- bert_model.ckpt.meta
```

```
|_____ vocab.txt
```

The input in this case is just a file with one sentence per line. Let us call it sentences.txt and put it into our \${CLASSIFIER_DATA} folder. You can generate the embeddings from the last hidden layers by identifying them as -1 (last hidden layer), -2 (hidden layer before that), and so on. The command to extract BERT embeddings for your input sentences is as follows:

```
$ export BERT_BASE_DIR = ./data/uncased_L-12_H-768_A-12
```

```
$ export CLASSIFIER_DATA = ./data/my_data
```

```
$ export TRAINED_CLASSIFIER = ./data/my_classifier
```

```
$ python extract_feature.py \
```

```
    --input_file=${CLASSIFIER_DATA}/sentences.txt \
```

```
    --output_file=${CLASSIFIER_DATA}/embeddings.json1 \
```

```
    --vocab_file=${BERT_BASE_DIR}/vocab.txt \
```

```
    --bert_config_file=${BERT_BASE_DIR}/bert_config.json \
```

```
    --init_checkpoint=${BERT_BASE_DIR}/bert_model.ckpt \
```

```
    --layers=-1, -2, -3, -4 \
```

```
    --max_seq_length=128 \
```

```
    --batch_size=8
```

The command will extract the BERT embeddings from the last four hidden layers of the model and write them out into a line-oriented JSON file called embeddings.json1 in the same directory as the input

file. These embeddings can be used as input to downstream models that specialize in some specific task, such as sentiment analysis. Because BERT was pretrained on large quantities of English text, it learns a lot about the nuance of the language, which turn out to be usefful for these downstream tasks. The downstream model does not have to be a neural network, it can be a non-neural model such as SVM or XGBoost as well.

There is much more you can do with BERT. The previous use case corresponds to transfer learning in the computer vision. As in computer vision, it is possible to fine-tune BERT (and other transformer models) for specific tasks, where the appropriate “head” network is attached to BERT, and the combined network is fine-tuned for a specific task.